Coding:
Verification, size measures, complexity analysis, coding standards, Effort Estimation, Cost Estimation, Testing fundamentals
Construction of the system: traditional and incremental approaches, conversion methods, Software Implementation, Overview of maintenance process, types of maintenance.
****

Verification:

Code verification is the process of checking the software code for errors which is introduced during coding phase. This process includes checking the consistency of user requirements within the design phase. This process does not verifying the correctness of programs. It verifies whether the software code has been translated as per user requirements.
The code verification techniques are classified into dynamic and static.

- The dynamic technique is performed by executing some test data. The outputs of the program are tested to find errors in the software code. This technique follows the conventional approach for testing the software code.
- In the static technique, the program is executed conceptually and without any data. A static technique does not use any traditional approach. Some of the commonly used static techniques are code reading, static analysis, symbolic execution, and code inspection and reviews.

Complexity analysis

It focus on designs and actual code. There is a direct correlation between design complexity or design errors, and code complexity or hidden defects. By identifying the properties of each their complexity, and identify those high risk applications that either should be revised or additional testing. Those software properties which is correlate to how complex such as size, interfaces among modules. Complexity metrics helps to determine the number of testing is needed to cover the design or coded logic.

Coding standards

Coding is the process of translating software design specification into source code through specific programming language. Coding is done by the coder or programmers who are independent people than the designer. Efficient coding reduce cost of testing and maintenance.

Coding standards are the set of guidelines or rules that are globally set by various software development organizations and these guidelines should be followed by all the developers to produce a good quality software.

Benefits of coding standards-

- Easy to understand
- Entire code of software has same format and style
- Developer dependency of code is reduced
- It improves readability, and maintainability of the code and it reduces complexity.
- It helps in code reuse and detecting errors easily in early phase.
- Encourages good programming skills and increases efficiency of the programmers.
- It reduces the hidden cost for developing the software.
- Increase the efficiency of the software and reduces the development time.

General coding standards

1. **Standard headers for different modules:**
   For better understanding and maintenance of the code, the header of different modules should follow some standard format and information. The header format must contain below things:
   - ✓ Name of the module
   - ✓ data of creation, date of approval,
   - ✓ developer's or author's name, synopsis,
   - ✓ description of the module,
   - ✓ description of the variables and functions defined in the module,

2. **Naming conventions for local variables, global variables, constants and functions:**
   Meaningful and understandable variables name helps anyone to understand the reason of using it.
   - ✓ Local variables should be named starting with small letter (e.g. localData)
   - ✓ Global variables names should start with a capital letter (e.g. GlobalData).
   - ✓ Constant names should be formed using capital letters only (e.g. CONSDATA).
   - ✓ It is better to avoid the use of digits in variable names.
   - ✓ The names of the function should be written in small letters.
   - ✓ The name of the function must describe the reason of using the function clearly and briefly.
3. **Indentation:**
   Proper indentation is very important to increase the readability of the code. For making the code readable, programmers should use White spaces properly. Some of the spacing conventions are given below:
   - ✓ There must be a space after giving a comma between two function arguments.
   - ✓ Each nested block should be properly indented and spaced.
   - ✓ Proper Indentation should be at the beginning and end of each block in the program.
   - ✓ All braces should start from a new line
4. **Error return values and exception handling conventions:**
   All functions that encountering an error condition should either return a 0 or 1 for simplifying the debugging.
5. **Avoid using a coding style that is too difficult to understand:**
   Code should be easily understandable. The complex code makes maintenance and debugging difficult and expensive.
6. **Avoid using an identifier for multiple purposes:**
   Each variable should be given a descriptive and meaningful name indicating the reason behind using it. This is not possible if an identifier is used for multiple purposes and thus it can lead to confusion to the reader. Moreover, it leads to more difficulty during future enhancements.
7. **Code should be well documented:**
   The code should be properly commented for understanding easily. Comments regarding the statements increase the understandability of the code.
8. **Length of functions should not be very large:**
   Lengthy functions are very difficult to understand. That's why functions should be small enough to carry out small work and lengthy functions should be broken into small ones for completing small tasks.
9. **Try not to use GOTO statement:**
   GOTO statement makes the program unstructured, thus it reduces the understandability of the program and also debugging becomes difficult.

📣 **Size Measures (Size Estimation):** Two techniques are used to estimate project size.

1. Lines of code or LOC
2. Function point

1. Lines of code or LOC

LOC count the total number of lines of statements or instructions written in the source code of software project. LOC is used to predict the amount of effort that will be required to develop a program as well as estimate programming productivity. It is also used for calculating productivity of programmers.

- Count in LOC-
  Declarations, Actual Code including logic and computation.

Ex:
```
for (i = 0; i <100; i += 1)
{
    Printf ("hello");
}  /* How many lines of code is this? */
```

- 4 Physical Lines of Code (LOC)
- 2 Logical Line of Code (LLOC)

- 1 comment line

- Don't count in LOC-

  Blank Lines: used to improve the readability of code

  Comments: used to help in code understanding as well as maintaince

  Ex: for (i = 0; i < 100; i += 1) printf("hello"); /* How many lines of code is this? */

  Ambiguity meets in above LOC:

The units of LOC are:

- KLOC- Thousand lines of code
- NLOC- Non comment lines of code
- KDSI- Thousands of delivered source instruction

Advantages:

- Estimation is closer to developer's perspective.
- Simple to use.

Disadvantages:

- It is language dependent
- No proper industry standard exist for this technique.
- It is difficult to estimate the size using this technique in early stages of project.

2. Functional Point (FP) Analysis

Allan J. Albrecht initially developed function Point Analysis in 1979 at IBM and it has been further modified by the International Function Point Users Group (IFPUG). The Function Point Analysis technique is used to analyze the functionality delivered by software to user. FPA is measuring software productivity and the cost associated with the development and maintenance.

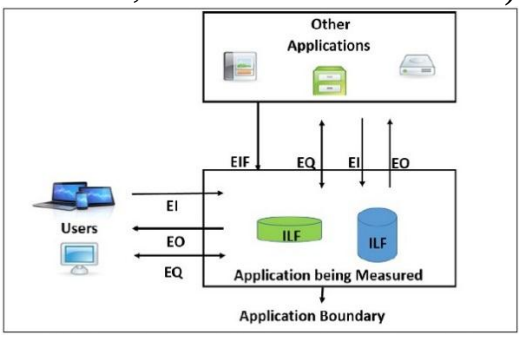
The units of FPA: *Unadjusted Function Point (UFP)*

➢ **Objectives of FPA**

1. To measure functionality that the user requests and receives.
2. It analyses S/W development and maintenance independent of the implementation technology.
3. To minimize the overhead of the measurement process.
4. FP describes the software complexity and shows project timelines and labour requirements.
5. It should be a consistent measure among various projects and organizations.

Types of FPA (Measurements Parameters of FPA):

1. **Transactional Functional Type**
   - **External inputs**- data or control inputs (I/P Screen, files, tables, forms, messages) to the system
   - **External outputs-** data or control outputs from the system (Output screens and reports)
   - **External inquiries-** I/O queries which require a response (prompts, interrupts, calls, etc.)
2. **Data Functional Type –**
   - **External interfaces -** libraries or programs which are passed into and out of the system (I/O routines, sorting procedures, math libraries, run-time libraries, etc.)
   - **Internal data files-** groupings of data stored internally in the system (entities, internal control files, Databases and directories)



Weights of Function Point Attributes			
Measurement Parameter	**Simple**	**Average**	**Complex**
External Inputs (EI)	3	4	6
External Outputs (EO)	4	5	7
External Inquiries (EQ)	3	4	6
Internal Logical Files (ILF)	7	10	15
External Interfaces (EIF)	5	7	10

*The functional complexities are multiplied with the corresponding weights against each function, and the values are added up to determine the UFP (Unadjusted Function Point) of the subsystem.

Function Point (FP)= Sum of Function point Count * CAF

where **CAF = [0.65 + 0.01 *∑(f_i)]**

$\sum(f_i)$ is the sum of all 14 questionnaires scale(0 to 5) shows the **complexity adjustment factor-CAF** (where i ranges from 1 to 14).

Example: Calculate the function point per function for software application with multiple Processing Factors 5, 1, 0, 4, 3, 5, 4, 3, 4, 5, 2, 3, 4, 2 by using following given Data:

1. The number of EI (Avg): 22,
2. The number of EO (Low): 45,
3. The number of EI (High): 06,
4. The number of ILF (Avg): 05,
5. The number of ELF (Low): 02.

Solution:

Measurement Parameter	Count Total Weighing factor
Number of external inputs (EI)	22*4 = 88
Number of external outputs (EO)	45*5 = 225
Number of external inquiries (EQ)	6*6 = 36
Number of internal files (ILF)	5*10 =50
Number of external interfaces (EIF)	2*5 = 10
Count-total	**409**

So sum of all f_i ($i \leftarrow 1$ to 14) = 5+1+0+4+3+5+4+3+4+5+2+3+4+2 = **45**

FP = Count-total * [0.65 + 0.01 *$\sum(f_i)$]

= 409 * [0.65 + 0.01 * 45]

= 409 * [0.65 + 0.45]

= 409 * 1.10 = **450**

Advantages:

- FPA measures the size of the solution instead of the problem
- It helps in estimating overall project costs, schedule, and effort
- It is independent of technology/programming language
- It helps in the estimation of testing projects
- Used for contract negotiations with business groups
- Used in the early stages of project planning.
- Used to compare different projects and technologies (database, language etc).

Disadvantages:

- It is not good for real time systems and embedded systems.
- Many cost estimation models like COCOMO uses LOC and FPC.

Differentiate between FP and LOC

FP	LOC
1. FP is specification based.	1. LOC is an analogy based.
2. FP is language independent.	2. LOC is language dependent.
3. FP is user-oriented.	3. LOC is design-oriented.
4. It is extendible to LOC.	4. It is convertible to FP (backfiring)

Cost Estimation

If developing a new software, The Software Cost Estimation is a process to estimate the how much resources, cost and time required for software project before the development starts i.e requirements of effort, development time and resources to complete the software project. Software cost estimates are based on past performance.

The cost estimation is a tool to estimate the planning, budgeting and resource utilization for the software projects. Before cost estimation for a software project, we will have known that

- What are the actual requirements for a project and its scope?
- What is the complexity of those requirements?
- The project is broken into small parts which are estimated cost and schedule individually.
- Cost driver factors that affect the development (like, product, project, personal & hardware).

These are the input to the cost estimation process. It provides Effort, Development Duration, and Resources.

- **Effort:** The amount of effort required to complete the development of software measures in Man-Months (MM).
- **Development Duration:** The time duration required to complete the development of a software i.e. total development time.
- **Resources:** The number of Manpower required for a software in terms of time to complete.

Cost estimation is a set of techniques and procedures used to find out the cost estimates. These estimation techniques compute the output from the given set of inputs.

- The inputs to the process of cost estimation are called as cost drivers
- The outputs are expressed in the form of efforts, duration, loading, or modified requirements.

➢ **Cost Estimation Models - S**tatic or Dynamic.

- Static model: a single variable is taken as a key element for calculating cost and time.
- Dynamic model: all variable are interdependent, and there is no basic variable.

➢ **Cost estimation Techniques:**

1. **Top-down:** based on historical data of similar projects and use historical data on revenue, cost, and other estimates.
2. **Bottom up:** Break down project into smaller manageable units.se present situation to estimate revenue and costs.
3. **Algorithmic (parametric) model:**
   This software cost estimation technique use the mathematical equations to perform the software estimation. The mathematical equations are based on historical data or theory. SLOC (source line of code), function points, and other cost drivers are the inputs. For most algorithmic model, calibration to the specific software environment can be performed to improve the estimation. Examples of the parametric models are COCOMO (COnstructive COst MOdel), COCOMO II, Putnam's software life-cycle model (SLIM).
   Advantages:
   - Generate repeatable estimations
   - Easy to modify input data
   - Easy to refine and customize formulas
   - Objectively calibrated to experience

   Disadvantages:
   - Unable to deal with exceptional conditions
   - Some experience and factors cannot be quantified
   - Sometimes algorithms may be proprietary
4. **Expert Judgment:**
   This technique captures the experience and the knowledge of the estimator who provides the estimate based on their experience from a similar project to which they have participated. Examples: Delphi and Work Breakdown Structure (WBS).
   Advantages:
   - Useful in the absence of quantified, experimental data.
   - Differences between past project experiences and requirements of the proposed project
   - Can factor in impacts caused by new technologies, applications and languages.

   Disadvantages:
   - Estimate is only as good expert's opinion
   - Hard to document the factors used by the experts
5. **Estimation by analogy:** This technique used when limited project information is available. This technique does not provide a reliable estimation. You can estimate the cost of the project by comparing it with any similar past project or historical records. Then you will use expert judgment

to find your project's cost estimate. If your organization has completed many similar projects, you will select the one that most closely resembles yours. The estimate can be accurate if the similarity is high and the estimators are subject experts. Analogous estimating is also known as top-down estimating. The benefits of this technique are a quick result with less effort. This is the fastest but least accurate technique.

6. **Price to Win:**
   The cost estimate is the price that is necessary to win the contract or the project.
   Advantages: Often rewarded with the contract
   Disadvantages: Time and money run out before the job is done

COCOMO Model

COCOMO (Constructive Cost Model) is based on LOC, i.e **number of Lines of Code**. It is a procedural cost estimate model predicting size, effort, cost, time and quality. It was developed by Barry Boehm in 1981. It estimates the required number of Man-Months (MM) for development of software products.

Software projects are classified into three categories:

1. **Organic –** It belongs to small and simple software projects which are handled by a small team with nominal experience, good knowledge and few fixed requirements.
   Example: Small data processing or Inventory management system.
2. **Semi-detached –** The team-size is medium with mixed experience (experience & inexperience resources) to deals with fixed and changeable requirements. The projects are difficult to develop. It also requires better guidance, more experienced developers.
   Example: Database design or OS development.
3. **Embedded –** Team size is larger than other two models. A software project requires the highest level of complexity, creativity, and experience. Software projects must be developed within set of tight hardware, software and operational Constraints.
   Example: Banking software or Traffic light control software.

➢ **Types of COCOMO Models:**

1. Basic COCOMO Model
2. Intermediate COCOMO Model
3. Detailed COCOMO Model

1. Basic COCOMO Model:

It is the one type of static model to calculate a quick estimation of software development costs and effort roughly. This model is based on Lines of code. Less accuracy of estimation because we don't consider the all parameters belongs to the project.

Estimation of Effort: **Effort (E) = a*(Size)^b Man-Months (MM)**

Time (D) = c*(Effort)^d Months(M)

Required average staff (Resources)= Effort / Time Person

Productivity (P) = KLOC / Effort

- E = Total effort required for the project in Man-Months (MM).
- D = Total time required for project development in Months (M).
- Size = the size of the code in thousands line of code KLOC.
- a, b, c, d = The constant parameters for a software project.

The coefficient a,b,c & d are constant and can be calculated using the given table:

Project	a	b	c	d
Organic	2.4	1.05	2.5	0.38
Semi-detached	3.0	1.12	2.5	0.35
Embedded	3.6	1.20	2.5	0.32

Example 1: Assume that the size of an organic type software product has been estimated to be 32,000 lines of source code. The average salary of software engineers be Rs. 15,000/- per month. Determine the effort required to develop the software product and the nominal development time.

Solution: As per the basic COCOMO estimation formula for organic software:

Effort = 2.4 * $(32)^{1.05}$ = 91 PM

Time = 2.5 * $(91)^{0.38}$ = 14 months

Cost required to develop the product = 14 * 15,000 Cost = Rs. 2,10,000/-

Example 1: A project was estimated with a size of 300 KLOC. Calculate the Effort, Scheduled time for development. Also, calculate Productivity of the software for Organic project type.

Solution: Given estimated size of project is: 300 KLOC

For Organic

Effort (E) = $a*(Size)^b$

= $2.4*(300)^{1.05}$

= 957.61 Man-Months

Time (D) = $c*(Effort)^d$

= $2.5*(957.61)^{0.38}$ = 33.95 Months

Productivity of Software = KLOC/Effort

= 300/957.61 = 313 **LOC/MM**

2. Intermediate COCOMO Model –

It is an extension of Basic COCOMO model. This model estimates software development effort in terms of size of the program and 15 cost driver parameters (product parameter, hardware parameter, resource parameter, and project parameter) of the project. This enhances the accuracy of estimation.

Intermediate COCOMO formula estimated effort and time:

E= (a $(Size)^b$) * EAF Man-Months

Time (D) = $c*(Effort)^d$ Months(M)

* Constant values of a,b,c,d refer at basic cocomo model table.

Where, **EAF =** Effort Adjustment Factor calculated by multiplying the parameter values of different cost driver parameters. For ideal, the value is 1.

Classification of Cost Drivers and their attributes:

Cost Drivers	Very Low	Low	Nominal	High	Very High
Product Attributes					
Required Software Reliability	0.75	0.88	1.00	1.15	1.40
Size of Application Database		0.94	1.00	1.08	1.16
Complexity of the Product	0.70	0.85	1.00	1.15	1.30
Hardware Attributes					
Runtime Performance Constraints			1.00	1.11	1.30
Memory Constraints			1.00	1.06	1.21
Volatility of the virtual machine environment		0.87	1.00	1.15	1.30
Required turnabout time		0.94	1.00	1.07	1.15
Personnel attributes					
Analyst capability	1.46	1.19	1.00	0.86	0.71
Applications experience	1.29	1.13	1.00	0.91	0.82
Software engineer capability	1.42	1.17	1.00	0.86	0.70
Virtual machine experience	1.21	1.10	1.00	0.90	
Programming language experience	1.14	1.07	1.00	0.95	
Project Attributes					
Application of software engineering methods	1.24	1.10	1.00	0.91	0.82
Use of software tools	1.24	1.10	1.00	0.91	0.83
Required development schedule	1.23	1.08	1.00	1.04	1.10

Example: A given project was estimated with a size of 300 KLOC. Calculate the Effort, time for development by considering developer having high application experience and very low experience in programming.

Solution: Given the estimated size of the project is: 300 KLOC
Developer having highly application experience: 0.82 (as per above table)
Developer having very low experience in programming: 1.14(as per above table)
EAF = 0.82*1.14 = 0.9348

Estimation of development effort (E) = $a*(Size)^b*EAF$
= $3.0*(300)^{1.12}*0.9348$
= 1668.07 MM

Estimation of development time (D) = $c*(Effort)^d$
= $2.5*(1668.07)^{0.35}$
= 33.55 Months(M)

Example: database system for an office automation project. Project = organic (a=3.2, b = 1.05, c=0.38), 4 modules to implement: data entry 0.6 KLOC data update 0.6 KLOC query 0.8 KLOC report generator 1.0 KLOC System SIZE 3.0 KLOC. A EAF of Complexity high 1.15, storage high 1.06, experience low 1.13, programming capabilities low 1.17 How many people should be hired?

3. Detailed Model –

A Solution of basic and intermediate COCOMO models problem. A large systems are divided into several smaller sub-systems. This Model uses effort multipliers for each phase of the project. These phase dependent effort multipliers produce better estimates because the cost driver ratings may be different during each phase. In this model, cost of each subsystem is estimated separately. This approach reduces the margin of error in the final estimate.

The Six phases of detailed COCOMO are:

1. Planning and requirements
2. System design
3. Detailed design
4. Module code and test
5. Integration and test
6. Cost Constructive model

Example: A distributed Management Information System (MIS) product for an organization having offices at several places across the country can have the following sub-components:

- The Connectivity part - Embedded software.
- The database part - Semi-detached software
- The GUI part - Organic software.

The costs for these three components can be estimated separately, and summed up to give the overall cost of the system.

Advantages of COCOMO Model:

- Easy to estimate the total cost of the project.
- Easy to implement with various factors.
- Provide ideas about historical projects.

Disadvantages of COCOMO Model:

- It ignores requirements, customer skills, and hardware issues.
- It limits the accuracy of the software costs.
- It mostly depends on time factors i.e. not a perfect realistic model
- It is not suitable for non-sequential, rapid development, reengineering , reuse cases models.

Testing Fundamentals

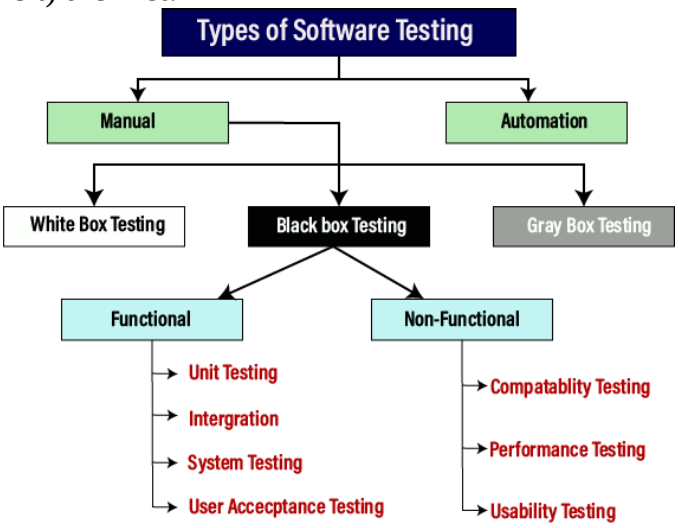
Testing is a group of techniques to determine the correctness of the application under the predefined conditions and data. It is a process of identifying the correctness of software by considering its all attributes (Reliability, Scalability, Portability, Re-usability, Usability) and evaluating the execution of software components to find the software bugs or errors or defects. It mainly aims at measuring specification, functionality and performance of a software program or application.

Testing necessary are:

- It measures the quality of the software.
- Gives confidence in the quality.
- Reduces the overall level of risks.
- Allows root cause analysis of bugs and defects.
- Helps measure the reliability of the software.
- Verifies and validates user requirements.

The three main goals of Testing are:

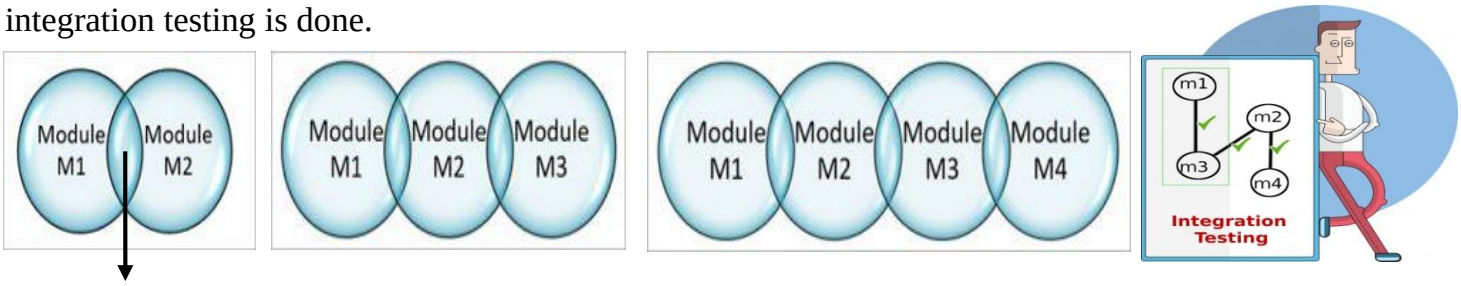
- Defect Detection: Find defects / bugs in the software during all stages of its development (earlier, the better).
- Defect Prevention: help anticipate and prevent defects from occurring at later stages of development or from recurring in the future.
- User Satisfaction: Ensure customers / users are satisfied that their requirements (explicit or implicit) are met.



Traditional and incremental approaches

Integration Testing

Most of the Software applications are dividing into many modules or units and developed by different development teams. Each module tests individually in unit testing phase. Instead of integrating all modules at once and make a system. We are integrated every modules incrementally i.e. one by one all the modules or components are added (integrated) logically and make a complete system and performing testing on the system to check the original functional requirements are implemented correctly or not. Adding the modules incrementally to ensure interface and interaction between modules and check the data flows between modules. After integrating different modules then the integration testing is done.



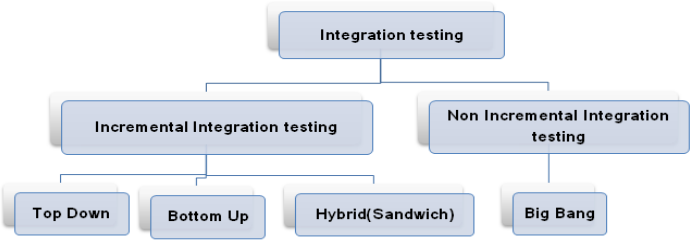
Integration Testing (In figure adding or integrate each module M1, M2,M3,M4 incrementally)

Integration testing will start after completion of unit testing. Integration testing is done by a specific integration tester or test team.

➢ **Objective of Integration Testing or Why Integration testing requires:**

- A Modules developed by a different developer whose understanding and programming logic may different from other developer. Integration Testing ensure that different modules work together successfully after integration
- Interfaces of the software modules with the database or External Hardware could be wrong.
- Issues can be fixed in early phase without rework and in less cost

- Finds the defects or bugs that occur when two or more modules integrated. If integration is not done properly, it will affect overall performance of software program.
- to check interface between integrated components is fine



Example of Incremental Integration Approach:

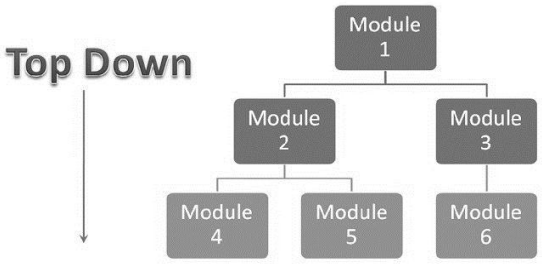
1. A Media player where all the modules like audio, video and various formats are combined and tested if the software is working correctly or not.
2. A bike assembling where all components are integrated one by one or by big bang method and then tested if they are working in a required manner.

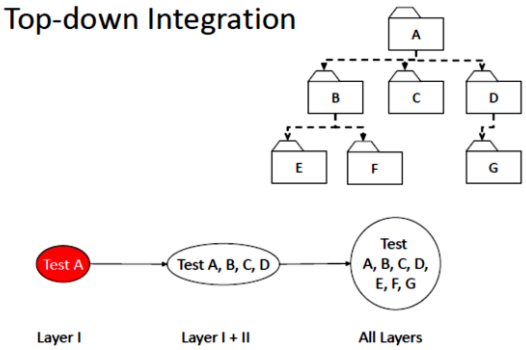
Stubs and drivers are pseudo code or dummy code used in Integration or component testing when one or more modules are not developed but are required to test some other module.

- **Stubs** are used in Top-down testing approach. Stubs help create the interface between lower lever modules which are not available or developed.
- **Drivers** are used in Bottom-up testing approach. Drivers help create the interface between top level modules which are not developed.

Top Down Incremental Approach

In top down integration testing, testing starts from top to bottom. Top level module is separately tested first and once they are tested successfully, then lower modules are integrated one by one. This process is repeated until all the modules are integrated and tested. The modules are added from the bottom.





Example

- Testing will start from module 1 then proceed to module 2, module 3 etc.
- In top down approach only module 1 is tested in isolation rest all other modules are integrated one by one and tested.
- Consider Module 2 is not developed yet so in order to test module 1 we need module 2, so here use stubs.
- So in place of module 2 stubs are placed. This stub will accept input from module 1 and returns the result. Stubs can be created by tester.

➢ **Advantages of Top down Approach:**

- Early introduction of design defects
- Main control points are tested early

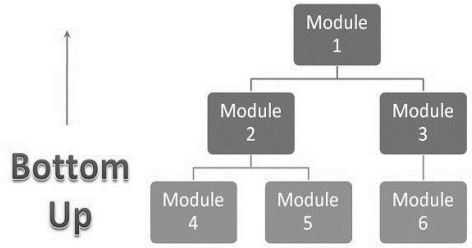
➢ **Disadvantages of Top down Approach:**

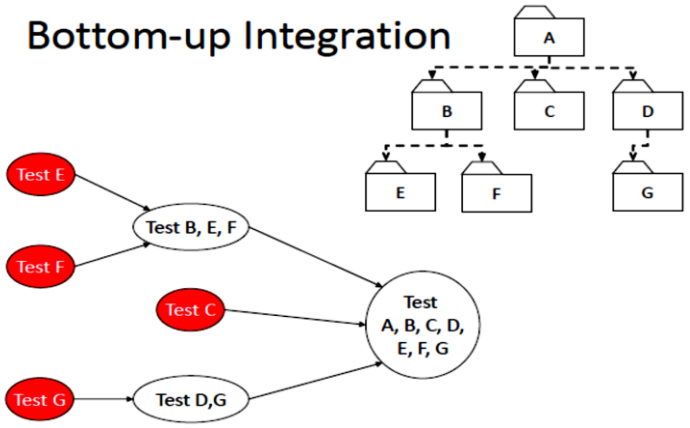
- Needs many Stubs. A stub is not a perfect implementation of related Module
- Modules at lower level are tested poorly.
- Significant modules are tested late in cycle
- It is very challenging to write test conditions

Bottom up Incremental Approach

In bottom up Approach testing, testing takes place from bottom to top, i.e. modules at bottom layer are integrated and tested first and then sequentially other modules are integrated as upwards to the

higher level modules. When higher level modules is missing or not developed then developer use temporary dummy program is known as 'DRIVER' i.e. unavailable or not developed modules are replaced by drivers.



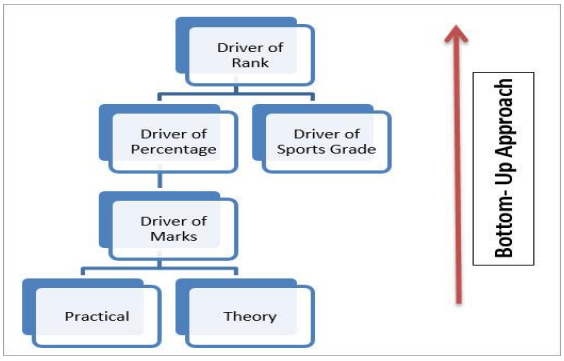


DRIVERS are the dummy programs which are used to call the functions of the lowest module in a case when the calling function does not exist.

Example: Testing will start from module 4, module 5 and module 6 and this process will continue till modules are covered i.e. the process will come to halt once module 1 gets tested.

- **Advantage of Bottom-Up approach:**
  - Identification of defect is easy.
  - Do not need to wait for the development of all the modules as it saves time.
- **Disadvantages of Bottom-Up approach:**
  - Drivers are more difficult to write than stub
  - Design defects are fixed in the later stage
  - Driver is not a complete implementation of related Module.

Example: Modules Rank, Marks, Percentage and Sports Grade are not yet developed so they will be substituted with related Driver:



Bottom up Incremental Integration testing approach
Following test will be derived:
1: Unit testing of module Practical and Theory
2: Integration and testing of Modules Marks-Practical-theory
3: Integration and testing of Modules Percentage-Marks-Practical-Theory
4: Unit testing of Module Sports Grade
5: Integration and testing of Modules Rank-Sports Grade-Percentage-Marks-Practical-Theory

Top- Down Approach	Bottom-UP Approach
Top-Down testing conducted from main-module to sub module.	Bottom-Up testing conducted from sub module to main module.
If sub module is not developed a temporary program called Stub is used for simulate the sub module.	If main module is not developed a temporary program called Driver is used to simulate the main module.
Top -Down testing good if major flaws occur toward the top of the program.	Bottom-Up Testing good if major flaws occur toward the bottom of the program.
In this test condition difficult to create.	In this test condition easy to create.
Observation of test output is more difficult.	Observation of test output is easier.

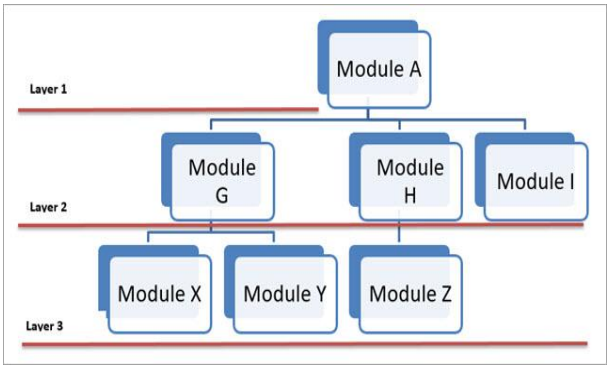
Hybrid (Sandwich) Approach

In this approach, both Top-Down and Bottom-Up approaches are combined for testing. Stub and drivers are used for incomplete or not developed modules.

A top-level modules are tested with bottom-up approaches and lower level modules tested with Top-Down approaches simultaneously. There is less possibility of occurrence of defect because each module interface is tested.

Testing Strategy:

1: Test A, X, Y, and Z individually – where Test A comes under Top layer test and Test X, Y and Z comes under Bottom layer tests
2: Test A, G, H and I
3: Test G, X, and Y
4: Test Hand Z
5: Test A, G, H, I, X, Y, and Z



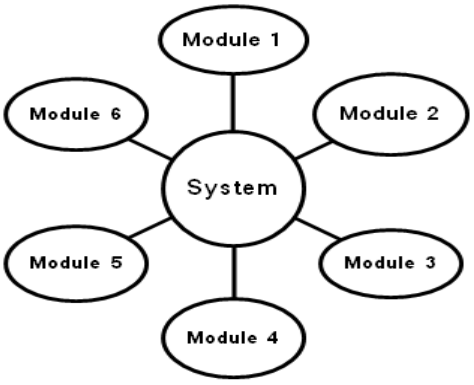
- **Advantages**
  - The hybrid method provides features of both Bottom Up and Top Down methods.
  - It is most time reducing method.
  - It provides complete testing of all modules.
- **Disadvantages**
  - This method needs a higher level of concentration as the process carried out in both directions simultaneously.
  - Complicated method.

Non Incremental Approach

When the data flow is very complex and when it is difficult to find who is a parent and who is a child. And in such case, we will create the data in any module bang on all other existing modules and check if the data is present. Hence, it is also known as the Big bang method.

Big Bang Approach:

- All the modules of the software are integrated simultaneously thus creating a big collection of modules and all the modules are tested together. Testing the modules are not tested individually.
- All the modules are integrated without performing any integration testing and then it's executed to know whether all the integrated modules are working fine or not.
- Because of integrating everything at one time if any failures occur then it became very difficult for the programmers to know the root cause of that failure.
- In case any bug arises then the developers has to detach the integrated modules in order to find the actual cause of the bug.



There are 6 modules named Module 1, Module 2, Module 3, Module 4, Module 5, and Module 6. These six modules are integrated simultaneously and all the six modules are tested all together. As these modules are not tested individually chances of occurring the failure may increase.

- **Advantages:**
  - This type of testing requires very little or no planning.
  - All the modules are completed before the integration testing starts.
- **Disadvantages:**
  - Since all the modules are tested together chances of failure increases.
  - Difficult to find the root cause of the failure if failure occurs.
  - Difficult to detach the modules if any error occurs.
  - Time consuming process.

Software Implementation and maintenance:

Systems implementation is the construction of the new system and its delivery into 'production' or day-to-day operation. Implementation is a process of ensuring that the information system is operational. It involves −

- Constructing a new system
- Constructing a new system from the existing one.

Implementation allows the users to take over its operation for use and evaluation. It involves training the users to handle the system and plan for a smooth conversion.

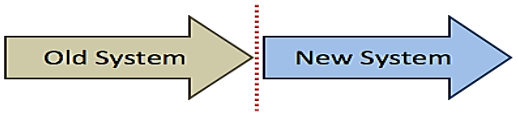
Conversion

The conversion is a process of replacing old existing system by new one. The all the data of old format is converted into new format. It provides understandable and structured approach to improve the communication between management and project team.

Conversion Methods-

1. Direct Changeover Conversion

An old system is directly replaced by the new system. It is the most risky conversion. It may be necessary when time is very short. It is also called crash conversion. The new system replaces entire existing system. The company stops using their old system and start using the new system.

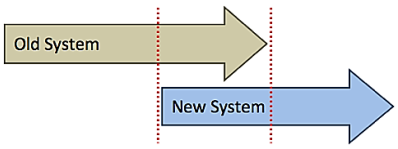


Example: a super market closes on a Friday; install all the software and hardware and re-opens on Monday with the new system in place.

Advantages	Disadvantages
▪ It is less costly as it is a direct change over ▪ It is not very time consuming as once the old system has stopped being used the new system is immediately being set up.	▪ Employees have to be trained before the implementation ▪ All the data will be lost if the new system fails and this will affect the whole organization. ▪ It is very difficult to detect small errors in the new system

2. Parallel Conversion

A type of conversion in which both new and old systems operate together for a period of time is called **parallel conversion.** It is the safest type of conversion. The results of both systems are compared. If the new system is successful they stop using the old system.



Example-an office prepares their payroll manually as well as using the payroll preparation system

Advantages	Disadvantages
▪ If the new system fails there will be no data lose because we have old system data ▪ Employees must be given training before implementation ▪ Small minor errors can be easily detected. ▪ Fixing any problems with the new system before ending previous system	▪ same work twice ▪ Delay in service, Customers unhappy. ▪ costly ▪ very time consuming

3. Phase-In Method

A type of conversion in which individual components of new system are implementation one by one is called phased conversion. Phased-In method introduces the new system to one department at a time.

Advantage	Disadvantages
▪ If the new system fails entire system is not affected ▪ More users friendly. ▪ Employees can be given training during the process of implementation. ▪ system is tested at every stage, very little chance of error	▪ Time consuming to implement. ▪ Costly

4. Pilot Approach

In this conversion, one part of the organization uses the new system and the rest of the organization uses the old system. When one part of the organization is satisfied with the new system, the rest of the organization can start using it.

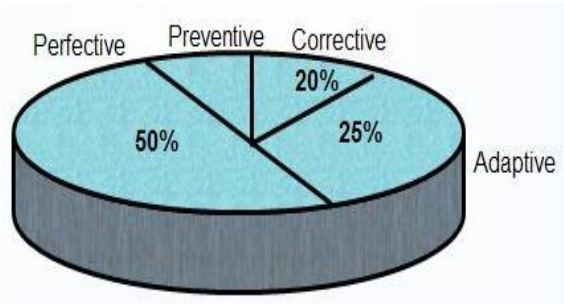
Advantage	Disadvantage
▪ Risk is reduced ▪ Allows the organization to see whether the new system will meet the organizations needs in one department/location before using it throughout the entire organization	▪ It takes long time to run the pilot, which leads to higher cost and time taken up by staff to evaluate the system

Software Maintenance:

When new system is installed, the periodic maintenance is needed to restore system performance in original condition. System maintenance is an ongoing activity, which covers variety of activities, including removing program and design errors, updating documentation and test data and updating user support. Maintenance means restoring system to its original conditions. Thus, maintenance changes the existing system, adding new features, enhance and modifying the code.

Need of software maintenance:

1. User requirements may changes.
2. Fix bugs and errors
3. Change in user requirement with time
4. Changing hardware/software requirements
5. To improve system efficiency
6. To optimize the code to run faster
7. To modify the components
8. To reduce any unwanted side effects.
9. Improve its existing features
10. Enhance its performance (speed, functionality)



- **Types of Maintenance**

Corrective Maintenance	Repair design and programming errors (bugs)
Adaptive Maintenance	Adapting the change in environment (different computers or different OS).
Perfective Maintenance	enhancing the system to meet the new requirements
Preventive Maintenance	changes made to improve future maintainability

- **Corrective maintenance:** Corrective maintenance is done to correct the detected problems or errors in different phases of the software system. The corrective maintenance is performed to restore or recover a system to a stable working condition after any performance failure occurs. This failure or error may be a logical error, coding error or may be any kind of design error.
  Corrective maintenance is a quick maintenance process when any failure is detected in the system. This immediate maintenance procedure is also known as fixing. Fixing is done in a short amount of time, So fixing may rise some unexpected error in the future if it is not done properly and carefully.
- **Adaptive maintenance:** Adaptive maintenance basically deals with changing in program functions. It is performed when there are some external changes in the environment occurs. These external

changes may occur due to any hardware-software technology changes or any government rule, business policy changes.

Example: A particular software was developed to calculate the taxes on profits of a company after deducting the dividend on equity shares. But after some period, the government has issued a new rule that the dividend should be included in the company profit for the tax calculation.

So, to adopt these changes, the software should be updated and the changes should be made in a particular function. Adaptive maintenance is necessary.

- **Perfective maintenance:** This deals with the existing features of the application. It involves enhancing, modifying and deleting certain functionalities, as well as adding new ones it increase the system's performance. A software product needs maintenance to support the new features that the users want or to change different types of functionalities of the system according to the customer demands

  Example: In a sub-office, the official data sent to headquarters via magnetic media. But after the sub-offices and headquarters are linked via a leased line, now they can use leased lines to interchange data to headquarters. But in this situation, an existing software is needed to be enhanced to send the data via the leased line. Perfective maintenance is to be done to the existing software to enhance its performance but it is very time consuming and very costly.

- **Preventive maintenance**: Preventive maintenance involves performing activities to prevent the occurrence of errors. It tends to reduce the software complexity by improving program understandability and increasing software maintainability.

  Preventive maintenance follows:
  - Re-engineering: An old software having some old features and technology is re-engineered with new technology which is now up to date and can produce great performance than the old system.
  - Reverse engineering: it includes the tasks like – optimization of existing code, updating the software documentation or code re-structuring of the software modules.

  Its goal is to boost the scalability of your system by fixing latent faults before they turn into operational errors. Preventive maintenance accounts for only 5% of all the maintenance activities.

  Example: If the number of users reaches a certain limit, the current system unable to handle it. The result is system crash. To prevent this, you can add servers before it happens. Servers are prepared to handle the increased traffic.

Software Quality Assurance (SQA)

Its prime goal is to ensure quality. Software Quality Assurance is a process which works parallel to development of a software. It is a process which assures that all development processes, methods, activities are monitored and fulfilled against the defined standards. These defined standards could be one or a combination of any like ISO 9000, CMMI model, ISO15504, etc. Software Quality Assurance is a kind of an Umbrella activity that is applied throughout the software process.

- **Quality-** Quality of the software is checked to see if it meets the user's requirements and defects free.
- **Assurance-** ensuring the correctness of the results and security of the product works without any bug.

Elements of Software Quality Assurance:

1. Standards
2. Review & Audit
3. Testing
4. Analyzing Error
5. Change Management
6. Vendor Management
7. Security Management
8. Risk Management
9. Safety

Major SQA Activities:

1. **SQA Management Plan:** Make a plan how you will carry out the SQA throughout the project. Think which set of software engineering activities are the best for project. Check level of SQA team skills.
2. **Setting Check Points:** SQA team sets the checkpoints after specific time intervals in order to check the progress, quality, performance of software and whether the software quality work is done on time as per the schedule and documents.
3. **Multi testing Strategy:** Do not depend on single testing approach. Multiple types of testing should be performed on software product to covers all angles to ensure better quality.
4. **Measure Change Impact:** Defect reported by QA and fixed by the developer, it is very important to retest and verify whether the fixed defect does not introduce new defects in the working software. Developers to check for newly generated defects by the introduction of new functionality or the fix of any defect.
5. **Maintaining Records and Reports:** It is important to keep all the records and documents of the QA and share them on time to time to customers. The test results, audit results, review reports, change requests documentation, etc. should be kept for future reference.
6. **Managing Good Relations:** Managing good relations between the testers and developers. The main aim of both teams should be the delivery of good quality of projects with minimum risks of failure.

Advantage of SQA:

1. SQA produce high quality software and saves cost.
2. SQA is beneficial for better reliability and no maintenance for long time.
3. Supports Productivity and Efficiency
4. Boost Customer Satisfaction

Disadvantage of SQA:

1. Difficult to Implement
2. Time Consuming
3. High Cost

Goals, Attributes, and Metrics:

- **Requirement quality:** SQA must ensure that the requirements specified by the user are properly reviewed by the software developers. Software developers have properly analyzed the correctness, completeness of specified requirements as it has a strong influence on the quality of software.
- **Design quality:** Evaluate whether the design model follows the specified requirements to achieve quality.
- **Code quality:** The code of the software. It's related descriptive information must follow the local coding standards. It is the job of SQA to observe the quality of code.
- **Quality Control Effectiveness:** Software developers should use minimal resources and try to achieve high quality. The job of SQA is to analyze whether the resources are allocated in an effective manner.

Goal	Attribute	Metric
Requirement quality	Ambiguity	Number of ambiguous modifiers in the specified requirement.
	Completeness	Number of requirements that are neither announced nor determined yet.
	Understandability	Number of specified requirements that are not understood by developers.
	Volatility	User can request a number of changes in the requirements per time.
	Traceability	Number of requirements that cannot be traced to design code.

	Model clarity	Developers can design a number of UML models for the specified requirements. Each model would have a number of descriptive pages.
Design quality	Architectural Integrity	Developers must be able to assess the architectural elements to confirm the quality of the architectural model.
	Component completeness	The components of the architectural model are completely defined or not.
	Interface complexity	Number moves required to get the desired function.
	Patterns	Number of patterns used to design architectural model.
Code quality	Complexity	number of cyclomatic complexity in the code
	Understandability	Percent of internal comment in the code and the naming convention used while naming the variables.
	Reusability	Number of components in the code that can be reused.
QC effectiveness	Resource Allocation	Time spent by staff per activity.
	Completion rate	Actual vs. budgeted completion time
	Review effectiveness	See review metrics
	Testing effectiveness	Number of errors found and Effort required to correct an error

Software Review Techniques

Software Review is systematic inspection of a software by one or more individuals who work together to find and resolve errors and defects in the software during the early stages of SDLC. Software review helps to software engineers in validating the quality, functionality, features and components of the software and it makes sure that it meets the user requirements.

Objective of software review is:

1. To improve the productivity of the development team.
2. To make the testing process time and cost effective.
3. To make the final software with minimum defects.
4. To eliminate the insufficiencies.

Types of Software Reviews:

1. **Software Peer Review:** Peer review is the process of evaluating the technical content and quality of the product. It is usually conducted by author of the work along with some other developers. Peer review is performed to examine or resolve the defects in the software, whose quality is also checked by other members of the team.

Peer Review has following types:

1.1 Code walkthrough

Walkthroughs and formal inspections are group-oriented methods (involve multiple people in the review process).Walkthrough is not a formal process. Walkthroughs are less formal than inspections. A set of people Checks program code and raise questions to the author. The author explains the logic of the code and answers the questions. If the author is unable to answer some questions, then he takes those questions and finds their answers.

- Author guides to review team via document that fulfill the common understanding and collecting feedback.
- A review team doesn't require detailed study before meeting but authors are already prepared.
- It is useful for higher-level documents i.e requirement specification and architectural documents.

1.2. Code review

Code review is systematic examination (known as peer review) of source code. It is planned to find and fix errors in the initial development phase, improving the overall quality of software and the developer's skills. Reviews are vary from informal to formal review.

Informal Reviews:

- Informal reviews are applied in the early stages of the life cycle of the document.

- These reviews are conducted between two person team. In later stages more people are involved.
- The aim of informal reviews is to improve the quality of the document and help the authors.
- These reviews are not based on the procedure and not documented.

Formal reviews:
Formal reviews follow the formal process i.e reviews are well structured &managed.

1.3. Code Inspection (Formal inspection)
The focus of this method is to detect all faults, violations and other side-effects. This method increases the number of defects detected by

1. Demanding thorough preparation before an inspection/review
2. Enlisting multiple views
3. Assigning specific roles to the multiple participates
4. Going sequentially through the code in a structured manner.

A formal inspection should take place only when the author has made sure the code is ready for inspection by performing some basic desk checking and walkthroughs. When the code is in such a reasonable state of readiness, an inspection meeting is arranged.

The author or the moderator selects the review team. The chosen members have the skill sets to uncover as many defects as possible. In an introductory meeting, the inspectors get copies (hard copies or soft copies) of the code to be inspected along with other supporting documents such as the design document, requirements document, and any documentation of applicable standards.

There are four roles in inspection:

- **Author of the code-**The author also presents his view of what the program is planned to do along with any specific issues that he want inspection team to put extra focus on.
- **Moderator-** The moderator informs the team about the date, time & venue of the inspection meeting. The moderator takes sequentially program code, asking each inspector if there are any defects in that part of the code. If any of the inspectors raises a defect, then the inspection team reflects on the defect and, when agreed that there is a defect, classifies in two types — minor/major and systemic/mis-execution.
- **Inspectors-** The inspection team assembles at the agreed time for the inspection meeting (also called the defect logging meeting). These are the people who actually provides, review comments for the code. There are typically multiple inspectors.
- **Scribe-** who takes detailed notes during the inspection meeting and circulates them to the inspection team after the meeting.

Defects found in the inspection meeting and the author fixing these defects. In case the defects are simple, the team may optionally call for a review meeting to inspect the fixes to ensure that they address the problems. In any case, defects found through inspection need to be tracked till completion and someone in the team has to verify that the problems have been fixed properly.

Technical Review

- Technical review is a discussion meeting that focuses on technical content of the document. **It is a less formal review.**
- It is guided by a trained moderator or a technical expert.
- The goal is to evaluate the value of technical concept in the project environment.
- Build the consistency in the use and representation of the technical concepts.
- In early stages it ensures that the technical concepts are used correctly.
- Notify the participants regarding the technical content of the document.

Pair Programming: where two developers develop code together at the same platform.

2. **Software Management Review:** Software Management Review evaluates the work status. Decisions regarding downstream activities are taken.
3. **Software Audit Review:** Software Audit Review is a type of external review in which one or more critics, who are not a part of the development team, organize an independent inspection of the

software product and its processes to assess their compliance with stated specifications and standards. This is done by managerial level people.

Software Reliability

Software Reliability is the probability of failure-free software operation for a specified period of time in a specified environment. Software reliability can defined as the attribute of quality aspect like functionality, usability, performance, capability, maintainability, documentation etc.

Software reliability is the quality factors that focus on the following tasks.

- Fault detection and recovery of software in case of failure
- Review of software development process
- Error prevention of software
- Measuring and analyzing errors

Three approaches are used to improve the reliability of software:

- **Fault avoidance:** The design and implementation phase of the software development uses the process that minimizes the probability of faults before the software is delivered to the user.
- **Fault detection and removal:** Verification and validation techniques are used to detect and remove faults. In addition, testing and debugging can also remove faults.
- **Fault tolerance:** The designed software manages the faults in such a way that software failure does not occur. There are three aspects of fault tolerance.
  - ✓ **Damage assessment:** This detects the parts of software affected due to occurrence of faults.
  - ✓ **Fault recovery:** This restores the software to the last known safe state. Safe state can be defined as the state where the software functions as desired.
  - ✓ **Fault repair:** This involves modifying the software in such a way that faults do not recur.

Assessing Reliability and Availability

The ability to measure or predict reliability in software engineering focuses on-

- ✓ Understanding and making trade-offs between reliability and other software characteristics such as performance, cost, and schedule
- ✓ Tracking progress when testing is complete
- ✓ Determining the effect of using new methods to develop software.

- Mean Time Between Failures (MTBF): how long the system can operate between failures of critical part. i.e. average of times between successive failures.
- Mean Time To Repair (MTTR): average amount of time taken to repair the machine after a failure occurs.
- Mean Time To Failure (MTTF): describes how long the software can be used to calculate MTBF.

Software reliability MTBF = MTTF + MTTR.

Availability of software is defined as the probability of software that working according with the user requirements at a specified time. Availability is an indirect measure to determine maintainability.

Software Availability = (1 – MTTR / (MTTR + MTBF))* 100%.

What is Risk?

Risk is an uncertain events that occur in the future due to lack of information, control or time. A possibility of suffering from loss in software development process is called a software risk. Software risk increases production cost, development of poor quality software, project not completed on time. Example: if a developer take leave from project. Other person is recruited in his place and he doesn't work on the same platform. Whether they will be able to complete the project on time with objective. That is risk of schedule.

Types of Risks:

1. Project risks
2. Technical risks
3. Business risks

1. Project risks:
Project risks threaten the project plan if project risks become real, it is likely that project schedule will slip and increase development cost. Project risks identifying possible problems of financial, schedule, resource, customer, requirements and their impact on a software project. It is very hard to monitor and control a software project. Example: an experience team member of a project may leave the organization in the middle of the project.
2. Technical risks: Technical risks concern possible design, implementation, testing, and maintenance problems. Most technical risks occur due to the development team's insufficient knowledge about the project. Causes of technical risks are:

- The project requirements are changing continuously.
- Latest technology is not available.
- Project implementation is complex.
- Integration of modules has become difficult.

3. Business risks: This risks contain risks of building an excellent product that no one need, losing economic.

Risk Management

Risk Management is identifying and removing these problems before they can damage the project. A software project identify large variety of risks which might affected.
Risk management have two activities risk assessment and risk control. These activities identify the risks and determine ways to eliminate their impact.

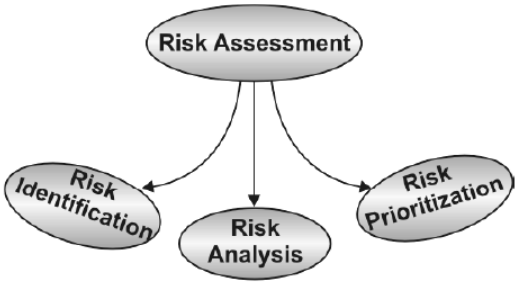
➢ Risk Assessment

Risk assessment focuses on determination of risks. The occurrence of risks depends on their nature, scope, and timing.

- ✓ **Nature of risks** specifies the problems that arise when risk occurs.
- ✓ **Scope of risks** specifies the capability of risks to impact the software.
- ✓ **Timing of risks** considers when and for how long the effect of risk is observed.

With the help of proper risk assessment, risks can be prevented. It determined the probability of existence of risks and its severity of risks (low, high, very high). Risk assessment can be carried out at any time during the project development but it starts from development process. A record of risk assessment should be maintained.
Risk assessment components:

- **Risk identification:** Identifies events that have an adverse impact on the project. These events can be a change in user requirements, new development technologies.



In order to identify the risks, project plan should be carefully analyzed in order to identify the areas of uncertainty before starting the project. There are various kinds of risks that occur during software development. Some of the common types of risks are-

- Technology risks: related to software or hardware technologies that are used in development
- People risks: connected with the person in the development team.
- Organizational risks: the organizational environment where the software is being developed.
- Tools risks: software tools and other support software used to create the system.
- Requirement risks: requirements change.
- Estimation risks: estimates of the resources required to build the system

- **Risk analysis:** After a risk is identified, it is evaluated to its impact on the project. Once the evaluation is done, risk is 'ranked' according to its probability of occurrence. After analyzing the

areas of uncertainty, a description of how these areas affect the performance of project is made. During the risk analysis process, you have to consider every identified risk and make a perception of the probability and seriousness of that risk.

- Probability of the risk determined- very low (0-10%), low (10-25%), moderate (25-50%), high (50-75%) or very high (+75%).
- Effect of the risk determined-catastrophic, serious, tolerable, or insignificant.

- **Risk prioritization:** Ranks of risks according to their capability to impact the project, which is known as **risk exposure**. Determination of risk exposure is done with the help of statistically-based decision mechanisms. These mechanisms specify how to manage the risk. A risks that impact on the project should be specified and eliminated completely. If the risks that are minor and have no or little impact on the project can be avoided.

➢ **Risk Control**

Risk control focuses on the management of risks in order to minimize or eliminate their effect. Risk control determines techniques and strategies to minimize the effect and resolve the factors that cause risks to occur. Once risk assessment is complete, risk control is performed to achieve an effective risk management.

- **Risk mitigation:** Minimizes the impact of risks, which are based on the occurrence of risks and their level of impact. Risk mitigation techniques acquire additional cost on extra resources and time, which are required to complete the project. To avoid this, a cost-benefit analysis should be done to evaluate the benefits of eliminating the risks or minimizing their impact along with the costs required to implement risk management. Appropriate measures should be taken before the risks occur, which later resolves the effect of risks when the project is in progress. It includes introducing measures and step taken into a project plan to mitigate, reduce, eliminate, or control risk. Risk mitigation means preventing risks to occur (risk avoidance).
- **Risk resolution:** Resolves the risks. Before resolving the risks, it is important to consider the areas of the project where risks can occur. After determining the areas, techniques are used for risk resolution. Generally, the techniques followed for risk resolution include cost/schedule estimates, quality monitoring, evaluation of new technologies, prototyping, and so on. The risks that have a very high impact should have an emergency plan. With the help of emergency plan, risks can be avoided or their impact can be minimized before they occur.
- **Risk monitoring:**
  The risk should be continuously monitored by reevaluating the risks, the impact of the risk, and the probability of the risk occurring.
  This guarantees that:
  - The dangers has been discovered and reduced.
  - The magnitude and impact of risk are assessed.

****

1. What is Risk? Explain Risk Management.
2. Explain formal and informal software Review Techniques.
3. Explain Function point and cocomo model with example.
4. What is Maintenance? Explain the types of Software Maintenance.
5. What is System Implementation? Explain Parallel Run, Pilot and Phase-In conversion methods with advantages.
6. Explain integration incremental testing approach.